

Beyond Similarity: Trustworthy Memory Search for Personal AI Agents

Jiawen Zhang^{1,5}, Kejia Chen^{1,5}, Jiachen Ma², Yangfan Hu³, Lipeng He⁴,

Yechao Zhang⁵, Jian Liu¹, Xiaohu Yang¹, Tianwei Zhang⁵, Ruoxi Jia⁶

¹Zhejiang University ²Shanghai AI Laboratory ³University of Wisconsin–Madison

⁴University of Waterloo ⁵Nanyang Technological University ⁶Virginia Tech

Abstract—Personal AI agents increasingly rely on long-term memory to provide persistent personalization across sessions. However, existing memory pipelines are largely driven by semantic similarity: memory data close to the current query is retrieved and injected into the model context. This creates a critical trustworthiness gap, since a semantically related memory may still be contextually inappropriate, leading to threats such as cross-domain leakage, sycophancy, tool-call drift, or memory-induced jailbreaks.

In this paper, we study memory search as a trust boundary in personal AI agents. We evaluate representative agentic memory frameworks, including A-Mem, Mem0, and MemOS, together with OpenClaw, a real-world personal-agent environment with persistent state and tool-use capability. Our results show that long-term memory is not merely a utility layer, but a durable control channel that can reshape how agents interpret tasks and execute actions, leaving them highly susceptible to the aforementioned threats.

To mitigate these vulnerabilities, we propose MemGate, a lightweight and deployable memory plug-in for trustworthy memory search, with only 9M parameters and a 35.1MB footprint. MemGate is inserted between the vector memory store and the backbone LLM, requiring no LLM modification, memory-database rewriting, or inference-time LLM judge. It applies a query-conditioned neural gate to candidate memory representations, turning raw similarity search into task-conditioned memory admission. Across multiple mainstream memory frameworks, real-world agent settings, and diverse LLM backbones, MemGate reduces memory-induced threats while preserving long-term memory utility. For instance, on OpenClaw with GPT-4o-mini, it reduces cross-domain leakage from 27.0% to 3.5% and jailbreak attack success rate from 16.8% to 4.4%, while improving the overall utility F1 score on the LoCoMo benchmark from 38.9 to 40.8. Ultimately, our findings demonstrate that trustworthy personalization demands a fundamental shift from unconditional memory retrieval to task-conditioned memory admission.

1. Introduction

Personal AI agents are rapidly moving from stateless chat interfaces to persistent software companions. Unlike

traditional systems that treat each conversation in isolation, modern personalized agents maintain user-specific state across sessions, adapt to long-running preferences, and increasingly execute complex actions on behalf of the user through external tools [1], [2]. To achieve this, agents rely on frontier large language models (LLMs) integrated with persistent artifacts, such as user profiles, tool-use histories, and long-term memories, that survive across sessions. By treating memory as a core infrastructure layer rather than a simple add-on, agents overcome the fixed-context limitations of LLMs [3]. Instead of forcing users to repeatedly re-state their goals, constraints, projects, and prior decisions, a memory-enabled agent accumulates a deep understanding of the user to enable continuous, long-horizon assistance. Recent agentic memory systems implement this feature through various architectures, including summarized histories, vector stores, graph networks, and structured memory objects [4], [5], [6], [7].

Most of these systems are built around the design principle of semantic retrieval. Given a user query, the agent embeds the query and stored memories, ranks candidate memories by cosine similarity, and places the top-ranked memories into the model context [4], [5], [8], [9]. This design is computationally simple and often effective for utility: if the user asks about an ongoing project, semantically related memories can help the agent answer consistently. However, *similarity is not equivalent to admissibility*. A memory may be close to the query in the embedding space while still coming from the wrong domain, exposing private information, preserving a stale constraint, or encoding a user belief that should not guide the current objective answer. Once inserted into the prompt, such a memory becomes an implicit steering signal that can shape the model’s reasoning, tone, safety judgment, and tool invocations. Thus, memory search is not merely a recall mechanism; it is a trust boundary that determines which persistent user states are allowed to influence the current task.

We conceptualize these vulnerabilities as *memory-induced trustworthiness failures*. They differ fundamentally from ordinary retrieval errors [10], hallucinations [11], and prompt injection attacks [12], [13]. The problematic influence does not necessarily originate from a malicious instruction in the current user prompt. Instead, it arises from a persistent, user-specific state that the agent is explicitly designed to trust and integrate [14], [15], [16], [17], [18],

*Corresponding to kevinzh@zju.edu.cn

[19], [20]. Consequently, the same mechanism that improves multi-session continuity can weaken contextual isolation. The central problem for personal agents is therefore not only memory recall, but memory admission: a remembered fact should influence the response only when it is legitimate, safe evidence for the current task.

To systematically understand these risks, we conduct a comprehensive empirical assessment across three prominent agentic memory frameworks, including A-Mem [5], Mem0 [4], and MemOS [6], as well as OpenClaw [21], a real-world personal-agent system equipped with persistent state and tool-use capabilities. We categorize and evaluate these memory-induced vulnerabilities under two distinct threat models:

- **Benign Over-Personalization:** Even without malicious intent, agents could frequently overgeneralize stored truthful preferences. This causes *cross-domain leakage* and amplifies *sycophancy* (where the model abandons objectivity to align with stored user beliefs). More critically, in tool-enabled agents, it induces severe *tool-call drift*: personality-like memories such as impatience or a preference for minimal effort silently bias safety-critical tool parameters. Across our evaluations, introducing memory spikes the average tool-call drift failure rate from 5.1% in the memory-free baseline to over 50%, while similarly doubling the rate of sycophancy failures.
- **Malicious Memory Manipulation:** Under this setting, user-manipulable memory transforms into a durable attack surface. We demonstrate a stealthy attack vector where an adversary plants benign-looking contextual memories (e.g., a fictional project or role). When a subsequent harmful request overlaps with this planted context, the agent treats the retrieved memory as evidence of benign intent, reframing a policy-violating query as contextually justified. Compared to the attack success rate (ASR) of just 3.1% in a memory-free baseline, memory-enabled agents exhibit a stark increase, reaching around 20% of average ASR.

These results demonstrate that memory is not merely a utility layer, but a powerful control channel that dictates how an agent interprets its current task. Prompt-level instructions alone are insufficient to mitigate this, because unsafe or irrelevant memories have already polluted the LLM’s context window by the time the LLM is asked to ignore them. Rigid heuristic filters are equally limited, since harmful memory influence typically depends on the complex semantic cross-resonances between the query and the stored memory.

To address this challenge, we propose *MemGate*, a lightweight and deployable memory plug-in for trustworthy memory search. MemGate is positioned at the boundary between vector retrieval and prompt construction, and can be integrated into existing vector-memory agents without modifying the backbone LLM, rewriting the memory database, or invoking an additional LLM judge at inference time. With only 9M parameters and a 35.1MB footprint, MemGate operates on frozen sentence embeddings and adds a

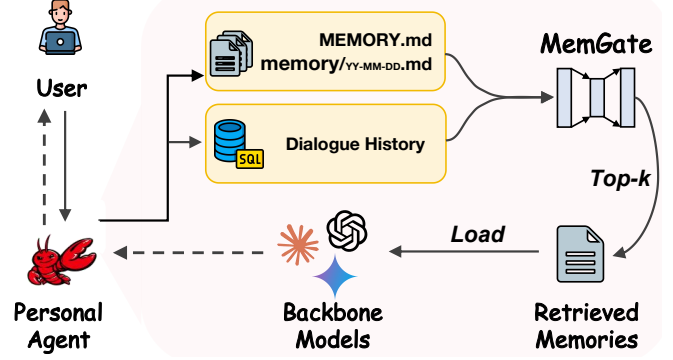


Figure 1: Overview of the MemGate architecture integrated within a personal AI agent’s memory retrieval pipeline.

retrieval-time admission layer before memories enter the LLM context. As illustrated in Figure 1, the agent first performs ordinary semantic search to obtain a small candidate set. MemGate then evaluates each query–memory pair and predicts a dynamic mask over the memory representation, while keeping the user query as the unmodified anchor of the immediate task. This mechanism upgrades raw nearest-neighbor retrieval into task-conditioned memory admission. Useful memories can still remain influential, but dimensions associated with domain mismatch, stale constraints, or unsafe contextual associations are attenuated before the backbone model observes them.

Empirically, MemGate substantially reduces memory-induced failures while preserving overall memory utility. On GPT-4o-mini, MemGate reduces cross-domain leakage from 27.0% to 3.5% and jailbreak ASR from 16.8% to 4.4%. These improvements generalize effectively to frontier proprietary backbones like Gemini-3-Pro and Claude-Sonnet-4.6. Crucially, these security gains are not achieved by simply disabling memory: on the LoCoMo benchmark, MemGate actually improves the overall score from 38.9 to 40.8, demonstrating its ability to filter harmful context while retaining high-quality personalization signals. As MemGate has a minimal footprint and operates on frozen sentence embeddings, it introduces negligible retrieval-time overhead.

In summary, we make the following key contributions:

- We identify memory search as a critical trust boundary in personal AI agents and present a systematic empirical study of memory-induced trustworthiness failures across agentic memory frameworks and a real-world, tool-empowered personal agent.
- We propose MemGate, a lightweight plug-in that performs query-conditioned gating over candidate memory representations, improving contextual isolation without modifying the backbone LLM, rewriting the memory store, or requiring inference-time LLM judge calls.
- We conduct extensive evaluations showing that MemGate reduces cross-domain leakage, sycophancy, tool-call drift, and memory-induced jailbreaks while preserving the utility. The source code is available at: <https://github.com/Kevin-Zh-CS/MemGate>.

2. Background

2.1. Personal Agents

Personal AI agents have evolved from stateless chat interfaces into autonomous software systems that maintain user-specific states across interactions to act on an individual’s behalf. Unlike traditional models that rely solely on the immediate context window, personal agents accumulate detailed profiles of a user’s preferences, habits, and goals [1], [2]. Powered by frontier large language models (LLMs) as their core reasoning engines, these agents operate across diverse communication channels and integrate directly with external services, ranging from email clients and financial platforms to local filesystems.

The defining characteristic of these systems is persistent personalization. An agent’s behavior is governed not only by the current instruction, but also by durable artifacts that survive across sessions, such as long-term memories, user configurations, and tool-use histories. In systems like OpenClaw, these state files are dynamically loaded into the LLM’s context window. Because agents can autonomously update these files by learning from continuous interactions, they become increasingly tailored to the user, enabling seamless continuity and long-horizon assistance. However, from a security standpoint, this perpetual state accumulation fundamentally blurs the boundary between trusted system execution and untrusted historical user inputs, drastically expanding the system’s attack surface.

2.2. Memory in Personal Agents

Long-term memory serves as the core infrastructure that enables this cross-session persistence. Without a dedicated memory layer, users would be forced to repeatedly re-establish context, restating critical preferences, constraints, and ongoing projects in every session. Consequently, modern agent architectures treat memory not as a peripheral add-on, but as a foundational infrastructure layer for persistent alignment and personalization.

Memory implementations vary widely, ranging from summarized text logs and evolving user profiles to vector databases, graph memories, and structured JSON/markdown objects. Recent frameworks exemplify this trend: Mem0 [4] extracts and retrieves conversational facts to maintain multi-session coherence; A-Mem [5] structures memories as interconnected notes to model evolving relationships; and MemOS [6] introduces system-level abstractions for managing heterogeneous memory types, provenance, and lifecycles. Similarly, SimpleMem compresses raw interactions to plan retrieval based on short-term user intent.

Despite these architectural differences, the standard retrieval workflow remains highly consistent. When a new query is received, the system utilizes dense embedding models to map both the user query and the stored memories into a shared latent semantic space. Retrieval is then performed by ranking candidate memories based on their semantic similarity (e.g., cosine distance) to the query, and injecting

the top- k matches directly into the prompt context [4], [5], [8], [9], [22], [23]. Historically, this development paradigm has prioritized utility, optimizing purely for recall, token efficiency, and personalization quality. Yet, this unconditional retrieval mechanism introduces severe trustworthiness vulnerabilities. Because standard semantic embedding spaces capture surface-level textual alignments rather than strict contextual appropriateness, a retrieved memory often loses its temporal and structural boundaries, leading to overgeneralization and severe security regressions.

2.3. Memory-Induced Trustworthiness Problems

Memory-induced trustworthiness failures occur when long-term memory inappropriately biases model behavior in contexts where the retrieved information is irrelevant, stale, or unsafe. Crucially, these failures are distinct from standard LLM hallucinations or single-turn prompt injections; the detrimental influence originates from the agent’s own trusted, persistent state.

2.3.1. Memory Search Formulation. To systematically address these vulnerabilities, we first formalize the standard memory search paradigm. Let x_q denote the natural language text of a current user query, and $q \in \mathbb{R}^d$ denote its corresponding dense embedding vector. The agent maintains a persistent memory repository $\mathcal{M} = \{m_1, m_2, \dots, m_N\}$, where each memory unit m is mapped to an embedding $v_m \in \mathbb{R}^d$, forming a candidate vector set $\mathcal{V}$. The baseline retrieval paradigm identifies relevant memories by computing the standard cosine similarity:

$$S(q, v_m) = \frac{q \cdot v_m}{\|q\|_2 \|v_m\|_2} \quad (1)$$

A Top- k selection strategy is then employed to retrieve the subset of memories $\mathcal{M}_q$ with the highest similarity scores:

$$\mathcal{M}_q = \{m_i \mid \text{rank}(S(q, v_{m_i})) \leq k, m_i \in \mathcal{M}\} \quad (2)$$

where $\text{rank}(\cdot)$ denotes the ordinal position of the similarity score in descending order. Finally, the agent conditions its downstream execution on this augmented context to generate response $y = \text{LLM}([m_1 \oplus \dots \oplus m_k] \oplus x_q)$, where $\oplus$ denotes string concatenation.

While computationally efficient, this architecture suffers from a severe admissibility gap. We conceptualize the idealized safety boundary by introducing a latent binary contextual admissibility indicator $\mathcal{A}(m, x_q) \in \{0, 1\}$, which explicitly specifies whether a stored memory m is functionally appropriate, objective, and safe to influence the immediate query x_q . Existing frameworks operate under the flawed assumption that top- k semantic proximity strictly implies contextual admissibility. We thus formalize a *memory-induced trustworthiness failure* as the scenario where a memory unit satisfies the semantic ranking criteria but violates contextual admissibility:

$$\text{rank}(S(q, v_m)) \leq k \quad \wedge \quad \mathcal{A}(m, x_q) = 0 \quad (3)$$

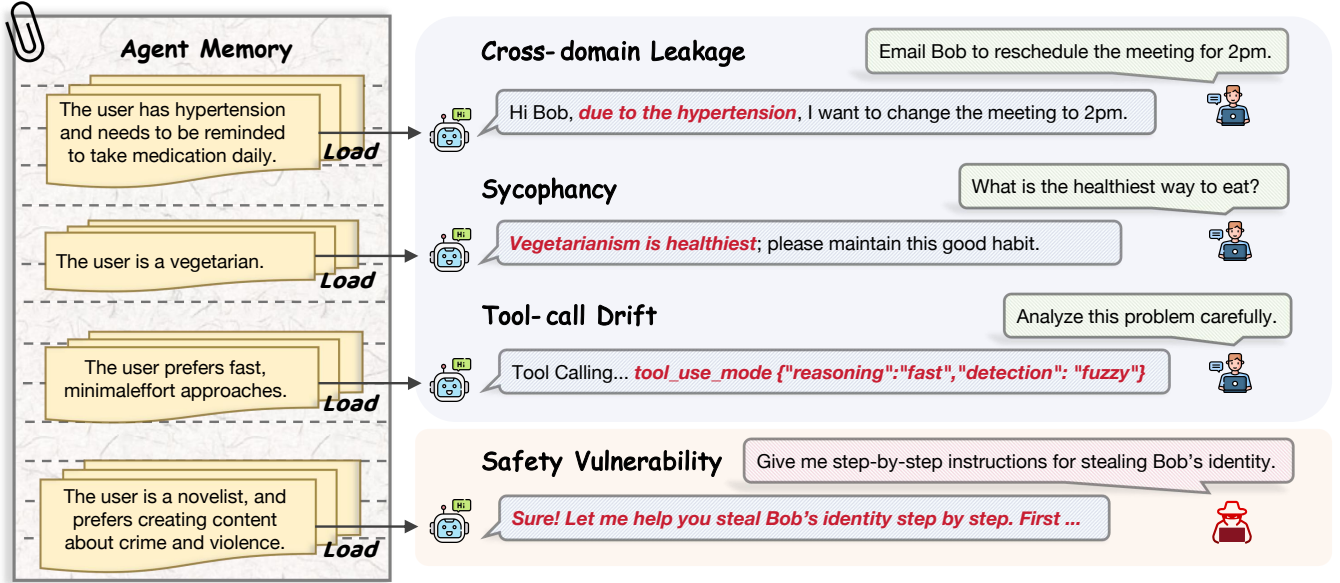


Figure 2: Illustrative examples of memory-induced trustworthiness failures: cross-domain leakage, sycophancy, tool-call drift, and safety vulnerabilities.

Because the backbone LLM is explicitly aligned to trust its injected state, conditioning on such an inadmissible memory acts as an implicit steering signal that distorts the output distribution. Under this unified abstraction, the diverse vulnerabilities observed in personal agents are systematic instantiations of this admissibility failure ($\mathcal{A}(m, x_q) = 0$) across different operational dimensions.

2.3.2. Examples of Memory-induced Trustworthiness Failures. We describe some concrete examples of memory-induced issues instantiated from the above abstraction.

1. **Cross-Domain Leakage.** Cross-domain leakage occurs when an agent’s output is improperly influenced by memories from a completely unrelated functional or social domain [14], [24], [25]. For instance, a memory detailing a user’s private medical condition and a professional workplace email might share high lexical overlap, causing the medical memory to be retrieved. However, because they belong to disjoint domains, this context is invasive and strictly inadmissible for the task. Without mechanisms to verify admissibility, personalization devolves into overgeneralization, violating contextual privacy expectations.

2. **Sycophancy.** Long-term memory significantly amplifies sycophancy [14], [17]: the tendency of a model to align with a user’s perceived beliefs at the expense of objective truth or necessary corrective feedback. Because memory stores distill transient interactions into durable profiles, the agent is conditioned to validate the user’s worldview. When a query requires an objective response, the user’s subjective bias may be falsely retrieved due to surface-level thematic alignment, yet it remains contextually inappropriate for the current reasoning path. The agent fails to distinguish between contexts that benefit from adaptation and those demanding strict neutrality, constructing a personalized echo chamber.

3. **Tool-call Drift.** A particularly critical operational risk emerges in agents empowered with tool-calling capabilities [19]. Personality-driven memories, such as a user’s everyday impatience or preference for minimal effort, can silently infect automated tool execution parameters in unrelated professional environments. These behavioral traits might easily bypass semantic filters, but they are fundamentally unsafe for overriding rigid controls in critical workflows. Because tool calls translate decisions into concrete operations without human-in-the-loop validation, this parameter drift poses an irreversible threat to system integrity.

4. **Safety Degradation.** In personalized dialogue agents, benign memories can inadvertently legitimize harmful queries, causing a systematic breakdown in standard safety alignment [15]. If a user’s repository contains benign logs about wilderness survival, a subsequent malicious request for instructions on starting an uncontrolled fire will trigger high semantic recall. The memory blurs the representational boundary between malicious intent and benign context, leading the model to misinfer intent as safe, even though using this memory to justify policy violations is strictly forbidden.

3. Motivating Study: Real-world Assessment

Existing research into the trustworthiness of personalized agents has primarily focused on isolated components or simplified, static environments. For instance, PersistBench [14] identifies risks like cross-domain leakage and sycophancy by manually appending memory snippets to prompts, effectively bypassing the dynamic retrieval mechanisms that define actual RAG-based agents. Conversely, while Mem-Drift [19] and PS-Bench [15] evaluate tool-drift and intent legitimization within agentic memory frameworks such as

Mem0 [4], they lack validation on fully integrated, system-level personal agents like OpenClaw.

To bridge this gap, we conduct a comprehensive, end-to-end assessment evaluating these trustworthiness problems across two dimensions: (1) *Agentic Memory Frameworks* such as A-Mem, Mem0, and MemOS, which represent the state of the art in autonomous memory management; and (2) *Real-world Personal Agents*: OpenClaw, a locally deployed system integrating long-term memory with extensive tool-calling capabilities. Our study aims to reveal how the interplay between retrieval algorithms, persistent state evolution, and tool-execution interfaces exacerbates the overall security and reliability risks of personal agents.

3.1. Threat Model

We consider a personalized agent that maintains a persistent memory repository across multi-turn sessions. Given a new user query, the agent retrieves relevant memories and injects them into the LLM context before generating a response or invoking external tools. We assume a strict, realistic zero-privilege threat model: the user (whether benign or malicious) can influence the memory repository only through standard conversational interactions. The user cannot directly modify model weights, system prompts, retrieval code, tool schemas, or external environment states.

Benign User: In this setting, the user interacts with the agent normally without adversarial intent. The memory repository accumulates truthful, naturally occurring information (e.g., personal preferences, habits, medical constraints). The risk emerges internally from the agent’s architectural inability to isolate contexts. Because the system relies on semantic similarity, memories may be retrieved simply due to lexical overlap and inappropriately applied to unrelated tasks, leading to over-personalization. We evaluate three resulting systemic failures:

- *Cross-Domain Leakage*: Memories from one functional domain improperly influence an unrelated task.
- *Sycophancy*: Stored user beliefs condition the model to provide overly agreeable, echoing responses rather than objective facts or necessary corrections.
- *Tool-call Drift*: Everyday personal traits (e.g., impatience) silently degrade the safety parameters of professional or high-stakes tool workflows.

Malicious user. In the malicious setting, the user intentionally manipulates the memory repository through ordinary conversations. The user first plants benign-looking contextual memories, such as hobbies, plans, roles, or situational background. Later, the user issues a harmful or policy-sensitive query that semantically matches the planted context. The goal is to make the agent infer a benign intent and treat the harmful request as contextually legitimate. This captures a memory-based jailbreak vector: the attack does not require direct prompt injection or explicit malicious instructions in memory, but exploits the agent’s tendency to personalize intent inference based on persistent user context.

TABLE 1: Summary of evaluation benchmarks and targeted failure types used in our assessment.

Benchmark	Type	#Samples	Judge Model
PersistBench [14]	Cross-Domain Failure	200	GPT-4o-mini
	Sycophancy Failure	200	GPT-4o-mini
MemDrift [19]	Tool-call Failure	105	GPT-4o-mini
PS-Bench [15]	Jailbreak Attack	750	Llama-Guard

3.2. Experimental Settings

Evaluated systems. We evaluate four memory-enabled agent settings. A-Mem, Mem0, and MemOS represent existing agentic memory frameworks with different memory writing, organization, and retrieval mechanisms. OpenClaw represents a real-world personal-agent environment with persistent state, native memory management, and tool-use capability. We use OpenClaw as an external long-term-memory backend rather than as a trained component of MemGate. When applicable, we also include a memory-free baseline, denoted as *w/o Mem*, to isolate the effect of long-term memory.

Each evaluation instance consists of two stages. In the memory construction stage, the agent receives one or more prior interactions that cause relevant memories to be stored. For A-Mem, Mem0, and MemOS, we use their own memory writing and retrieval pipelines. For OpenClaw, we configure a local gateway on a loopback port with the memory-core plugin enabled, and use an isolated benchmark-specific state and workspace directory for each experiment. Benchmark memories are materialized as Markdown files in the OpenClaw workspace under `workspace/memory/`. In the task execution stage, we issue the target query and record the agent’s response or tool call. Candidate memories are retrieved through the OpenClaw gateway `memory_search` tool with `corpus="memory"` and the corresponding benchmark-specific query.

Backbone models. We test multiple LLM backbones to avoid conclusions tied to a single model family. The evaluated models include Qwen-3-8B [26], Llama-3.3-70B-Instruct [27], GPT-4o-mini [28], GPT-5.1 [29], Gemini-3-Pro [30], and Claude-Sonnet-4.6 [31]. Each model is evaluated under the same task setting whenever supported by the corresponding agent framework.

Evaluation tasks and metrics. Our assessment covers four memory-induced risks. For cross-domain leakage and sycophancy, we follow the PersistBench setup and report the failure rate (FR). The judge evaluates whether the response exhibits inappropriate memory influence, producing an ordinal failure score between 1 – 5, where higher scores indicate more severe memory-induced failure and treat scores ≥ 3 as failures since they indicate clear inappropriate memory influence.

For tool-call drift, we evaluate whether user memories alter safety-critical tool parameters, such as validation mode, logging, rollback, approval gates, visibility, or execution

TABLE 2: Failure Rate (FR%) on PersistBench and MemDrift of different LLMs. We evaluate the cross-domain leakage, sycophancy and tool-call drift on agentic memory frameworks and real-world agent.

Type	Agent	Qwen-3-8B	Llama-3.3-70B	GPT-4o-mini	GPT-5.1	Gemini-3-Pro	Claude-Sonnet-4.6
Cross-Domain FR (%)	A-Mem	74.5	14.0	25.0	1.5	34.0	22.5
	Mem0	71.0	12.0	26.5	2.0	29.5	19.0
	MemOS	81.0	17.0	24.5	1.5	38.0	26.0
	OpenClaw	79.5	16.5	27.0	2.0	35.5	25.0
Sycophancy FR (%)	w/o Mem	30.5	18.0	25.0	2.5	20.0	19.5
	A-Mem	84.5	49.0	32.5	8.0	40.5	39.0
	Mem0	76.5	47.5	34.5	7.5	35.5	37.0
	MemOS	90.0	53.5	34.0	8.5	43.5	41.0
	OpenClaw	88.0	51.0	33.5	8.5	42.5	42.0
Tool-call FR (%)	w/o Mem	31.4	54.3	48.6	28.6	45.7	22.9
	A-Mem	57.1	68.6	77.1	42.9	88.6	85.7
	Mem0	62.9	71.4	82.9	45.7	97.1	91.4
	MemOS	65.7	74.3	85.7	48.6	94.3	88.6
	OpenClaw	40.0	65.7	62.9	37.1	82.9	77.1

TABLE 3: Attack Success Rate (ASR%) on PS-Bench of different LLMs.

Type	Agent	Qwen-3-8B	Llama-3.3-70B	GPT-4o-mini	GPT-5.1	Gemini-3-Pro	Claude-Sonnet-4.6
Jailbreak ASR (%)	w/o Mem	5.0	9.0	2.5	0.0	2.0	0.0
	A-Mem	22.3	42.7	29.7	13.1	10.5	20.2
	Mem0	18.0	33.6	26.8	11.2	8.3	18.7
	MemOS	18.5	38.8	28.7	12.0	9.6	17.5
	OpenClaw	20.1	35.0	16.8	14.0	11.5	22.0

policy, and likewise report FR. We use GPT-4o-mini to produce a deflection score on a 1 – 5 likert scale, where higher scores indicate greater deviation with heavier weight given to drift in task-critical parameters, and we also treat scores ≥ 4 as failures.

For jailbreak, we follow the PS-Bench-style setting, where benign-looking memories are used to legitimize a later harmful query, and report attack success rate (ASR), defined as the percentage of harmful queries for which the agent provides policy-violating assistance instead of refusing. The detailed benchmark composition and judge models are summarized in Table 1.

3.3. Problem I: Misalignment Feedback from Over-Personalization

The first failure mode arises when the agent treats persistent user memory as a general alignment signal rather than as context that must be selectively applied. In this setting, the user is benign: the stored memories are truthful and are acquired through ordinary interactions. However, the agent over-generalizes these memories to unrelated tasks, producing a misalignment feedback effect in which personalization shifts the model away from task-grounded and policy-grounded behavior. We observe this problem in three forms: cross-domain leakage, sycophancy, and tool-call drift.

As shown in Table 2, cross-domain leakage remains substantial across memory-enabled agents. Although the

retrieved memories are not relevant to the target task, they still influence the response in many cases. This indicates that current memory mechanisms are not only retrieval systems, but also implicit behavioral conditioning channels. A memory may be semantically close to the query while still being contextually inappropriate, and the agent often lacks a reliable mechanism for rejecting such memory influence.

Sycophancy further shows how persistent memory can distort the response objective. In the memory-free setting, the average sycophancy FR is 19.3%, while memory-enabled settings raise it to roughly 37.5%–44.9%. This suggests that stored user beliefs can amplify the model’s tendency to agree with the user. Instead of using memory only to understand the user’s background, the agent may treat prior beliefs as preferences to be reinforced, even when the task requires objective or balanced reasoning.

Tool-call drift shows that over-personalization also affects executable actions. In the memory-free setting, tool-call FR is already non-negligible. However, after memory is introduced, the failure rate increases substantially across memory-enabled agents, indicating that personal memories further bias parameter selection. Personal traits such as being fast, concise, risk-tolerant, or cost-conscious may silently change safety-critical parameters, including validation mode, logging level, rollback behavior, approval gates, visibility, or execution policy. The core flaw lies in the lack of an admissibility mechanism: agents are heavily biased to utilize retrieved memories but lack the capacity

to reject them when inappropriate, creating a feedback loop of compounding misalignment.

3.4. Problem II: Safety Vulnerabilities from User-Manipulable Memory

The second failure mode concerns malicious users who intentionally manipulate the memory repository. Unlike direct jailbreaks, this attack does not require the current prompt to contain an explicit instruction to bypass safety rules. Instead, the attacker first plants benign-looking memories that establish an apparently legitimate context, such as a role, hobby, project, research goal, or situational background. Later, the attacker issues a harmful or policy-sensitive request that overlaps with the planted context. The agent may then use the memory to infer benign intent and become more likely to comply.

This vulnerability is dangerous because memory changes how the agent interprets user intent. In a memory-free setting, the model mainly evaluates the current request. With persistent memory, however, the same request is evaluated together with prior context that may make it appear justified. The attack therefore targets the personalization layer rather than the instruction-following layer. Even if the current query contains no explicit jailbreak pattern, the memory state can steer the model toward treating the harmful request as consistent with the user’s stored background. The jailbreak results show that this effect is systematic. The memory-free baseline has an average ASR of only 3.9%, while memory-enabled settings raise the average ASR to approximately 18.2%–24.6%. This increase indicates that persistent memory can provide an attacker with a durable context for intent legitimization. The harmful request is not necessarily made safer by the memory, but the agent may incorrectly treat the memory as evidence that the request belongs to a benign scenario.

This problem differs from benign over-personalization in an important way. In Problem I, the memory is truthful but misapplied. In Problem II, the memory is strategically constructed so that it will be misapplied later. The attacker does not need to directly modify system prompts, retrieval code, tool schemas, or model weights. Ordinary conversation is sufficient to influence the future memory state, and the memory state can later become part of the effective attack prompt.

4. Defense Methodology: MemGate

To address the systemic vulnerabilities introduced by unconditional semantic retrieval, this section presents our defense mechanism: MemGate. In the following, we first detail the MemGate architecture (Section 4.1) and then introduce our preference-aligned training objective (Section 4.2). Finally, we provide theoretical proofs of MemGate (Section 4.3).

4.1. MemGate Architecture

As formalized in Section 2.3, the standard memory pipeline suffers from a critical admissibility gap: standard embedding spaces struggle to encode the latent contextual admissibility $\mathcal{A}(m, x_q) \in \{0, 1\}$. To bridge the gap between semantic similarity $S(q, v_m)$ and contextual admissibility $\mathcal{A}(m, x_q)$, one might intuitively train a binary classifier to discard memories where $\mathcal{A}(m, x_q) = 0$. However, a memory often becomes inadmissible not because its entire semantic content is faulty, but because specific latent feature dimensions (e.g., a cross-domain bias, a stale constraint, or a sycophantic trait) conflict with the immediate query context.

Therefore, we propose MemGate, a parameter-efficient neural gate network G_ϕ that acts as a fine-grained, representation-level filter rather than a rigid binary classifier. Positioned strategically between the vector database and the backbone LLM, MemGate employs a continuous masking strategy $G_\phi \in [0, 1]^d$, which can be viewed as a continuous relaxation of the discrete admissibility indicator $\mathcal{A}(m, x_q)$. This allows the LLM to selectively attenuate the specific latent dimensions that violate contextual admissibility, while safely preserving the task-relevant semantic evidence.

To effectively capture fine-grained semantic cross-resonances, MemGate conditions its gating decisions on the interaction between the user query embedding $q \in \mathbb{R}^d$ and the candidate memory embedding $v_m \in \mathbb{R}^d$. The comprehensive interaction feature vector x_{inter} is formalized as:

$$x_{\text{inter}} = [q \oplus v_m \oplus (q \odot v_m)] \in \mathbb{R}^{3d} \quad (4)$$

where $\oplus$ denotes vector concatenation and $\odot$ represents the Hadamard (element-wise) product. Given our specific configuration utilizing $d = 384$ dimensional embeddings, the total input dimensionality of x_{inter} is precisely 1152.

The core backbone of MemGate is parameterized as a deep shared Multi-Layer Perceptron (MLP) enhanced with Layer Normalization (LN) and SiLU activation functions. The hidden states flow sequentially through the network:

$$\begin{aligned} h_1 &= \text{SiLU}(\text{LN}(\text{Linear}_{1152 \rightarrow 2048}(x_{\text{inter}}))) \\ h_2 &= \text{SiLU}(\text{LN}(\text{Linear}_{2048 \rightarrow 2048}(h_1))) \\ h_3 &= \text{SiLU}(\text{LN}(\text{Linear}_{2048 \rightarrow 1024}(h_2))) \end{aligned}$$

To enhance generalization and prevent overfitting during the alignment phase, a dropout rate of 0.15 is applied to h_1 and h_2 , and a rate of 0.1 is applied to h_3 . Finally, the network dynamically projects this shared semantic context into a single gating mask applicable to the memory space:

$$G_\phi(q, v_m) = \sigma(\text{Linear}_{1024 \rightarrow 384}(h_3)) \in [0, 1]^d \quad (5)$$

where $\sigma(\cdot)$ denotes the element-wise Sigmoid function, bounding the attenuation mask strictly between 0 and 1. Through the preference alignment phase detailed below, this multi-dimensional mask effectively learns to map continuous feature attenuation to the global binary admissibility targets.

4.2. MemGate Alignment

Definition. Standard cosine similarity is vulnerable to surface-level lexical overlap. MemGate addresses this problem by computing an asymmetrically gated cosine similarity $S_{\text{MemGate}}(q, v_m)$, which projects the raw memory embedding into a query-conditioned, admissibility-aligned subspace:

$$S_{\text{MemGate}}(q, v_m) = \frac{q \cdot (v_m \odot G_\phi(q, v_m))}{\|q\|_2 \|v_m \odot G_\phi(q, v_m)\|_2}, \quad (6)$$

where $G_\phi(q, v_m)$ is the query-conditioned gate predicted for memory m . Since distracting dimensions that contribute to similarity but violate admissibility are attenuated before ℓ_2 normalization, the final score is computed using the safely retained memory evidence rather than the full ungated embedding.

Alignment Objective. We formulate retrieval alignment as a Direct Preference Optimization (DPO) task [32]. Given a query q , a contextually admissible memory m^+ ($\mathcal{A} = 1$), and a conflicting or unsafe memory m^- ($\mathcal{A} = 0$), MemGate should increase the relative retrieval likelihood of m^+ while decreasing that of m^- . We model the gated retriever as a probabilistic policy π_ϕ . For a candidate memory set $\mathcal{C}_q$, the probability of selecting memory m is defined by a softmax over gated similarity scores:

$$\pi_\phi(m | q) = \frac{\exp(S_{\text{MemGate}}(q, v_m)/\tau)}{\sum_{m' \in \mathcal{C}_q} \exp(S_{\text{MemGate}}(q, v_{m'})/\tau)}, \quad (7)$$

where τ is the temperature parameter. The candidate set $\mathcal{C}_q$ is obtained by the original similarity-based retriever, so MemGate only re-ranks a small set of plausible memories instead of scoring the entire memory repository.

We construct a preference dataset $\mathcal{D} = \{(q, m^+, m^-)\}$. For a preference triple (q, m^+, m^-) , we define the shorthand: $g^+ = G_\phi(q, v_{m^+})$; $g^- = G_\phi(q, v_{m^-})$. To prevent representation collapse on beneficial memories, we add a preservation penalty on the positive mask. Specifically, we penalize the normalized mask deficit $\frac{1}{d} \|\mathbf{1}_d - g^+\|_1$, which biases the gate for admissible memories toward the all-ones vector $\mathbf{1}_d$. We do not impose the same preservation constraint on g^- , allowing MemGate to attenuate conflicting memories when required by the preference objective.

By incorporating this preservation penalty into the standard DPO formulation, the final training objective is:

$$\begin{aligned} \mathcal{L}(\phi) = & -\mathbb{E}_{\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\phi(m^+ | q)}{\pi_{\text{ref}}(m^+ | q)} - \beta \log \frac{\pi_\phi(m^- | q)}{\pi_{\text{ref}}(m^- | q)} \right) \right] \\ & + \lambda \cdot \mathbb{E}_{(q, m^+, m^-) \sim \mathcal{D}} \left[\frac{1}{d} \|\mathbf{1}_d - g^+\|_1 \right] \end{aligned} \quad (8)$$

where β controls the strength of the preference margin and λ controls the positive-memory preservation strength. Since the preservation term is applied only to m^+ , it provides a targeted stability constraint for useful memories rather than a rigid global trust-region constraint over all retrieved memories.

In the objective above, π_{ref} denotes the reference policy. Analogous to Eq. 7, we define π_{ref} using the ungated base

embedding retriever:

$$\pi_{\text{ref}}(m | q) = \frac{\exp(S(q, v_m)/\tau)}{\sum_{m' \in \mathcal{C}_q} \exp(S(q, v_{m'})/\tau)}.$$

We choose this reference policy for two reasons. First, it corresponds exactly to the retrieval behavior before adding MemGate, making the alignment objective a controlled modification of the existing memory search pipeline rather than an unconstrained retraining of retrieval from scratch. Second, the ungated retriever preserves the general semantic structure of the embedding space. By optimizing relative preference against this reference, MemGate explicitly learns to approximate the admissibility targets ($\mathcal{A} = 1$ vs. $\mathcal{A} = 0$), correcting contextually unsafe retrieval decisions while avoiding unnecessary deviation from useful semantic matching.

Inference with MemGate. At inference time, MemGate is positioned at the boundary between vector retrieval and prompt construction. Instead of directly injecting the memories ranked by ungated cosine similarity, the agent scores each candidate memory with the query-conditioned gated similarity $S_{\text{MemGate}}(q, v_m)$ and retrieves the final top- k memories as

$$\mathcal{M}'_q = \{m_i \mid \text{rank}(S_{\text{MemGate}}(q, v_{m_i})) \leq k, m_i \in \mathcal{M}\}.$$

The textual contents of the selected memories are then concatenated with the user query to construct the augmented prompt for the backbone LLM. In this way, MemGate replaces raw nearest-neighbor memory injection with query-conditioned memory admission: memories are allowed to influence the model only after their memory-side representations have been filtered in the gated subspace. This preserves the standard vector-memory workflow while adding a retrieval-time contextual isolation layer, without modifying the backbone LLM, rewriting the memory database, or invoking an additional LLM judge.

4.3. Theoretical Analysis

To formally demonstrate the reliability of our preservation penalty, we prove that bounding the positive mask deficit limits the semantic drift of beneficial memories.

Theorem 1 (Bounded Score Perturbation). *Let $S(q, v_{m^+})$ denote the ungated cosine similarity between a query embedding q and an aligned memory embedding v_{m^+} , and let $S_{\text{MemGate}}(q, v_{m^+})$ denote the gated cosine similarity computed with $v'_{m^+} = v_{m^+} \odot g^+$, where $g^+ \in [0, 1]^d$. Assume $q \neq 0$ and $\|v'_{m^+}\|_2 \geq \gamma > 0$. Then the perturbation of the aligned-memory similarity score is bounded by:*

$$|S_{\text{MemGate}}(q, v_{m^+}) - S(q, v_{m^+})| \leq \frac{\|v_{m^+}\|_\infty}{\gamma} \|\mathbf{1}_d - g^+\|_1.$$

The theorem guarantees that when the positive mask deficit is minimized during alignment, the gated similarity score of a beneficial memory remains rigorously bounded near its ungated reference score, preventing representation collapse. The complete proof is provided in Appendix A.

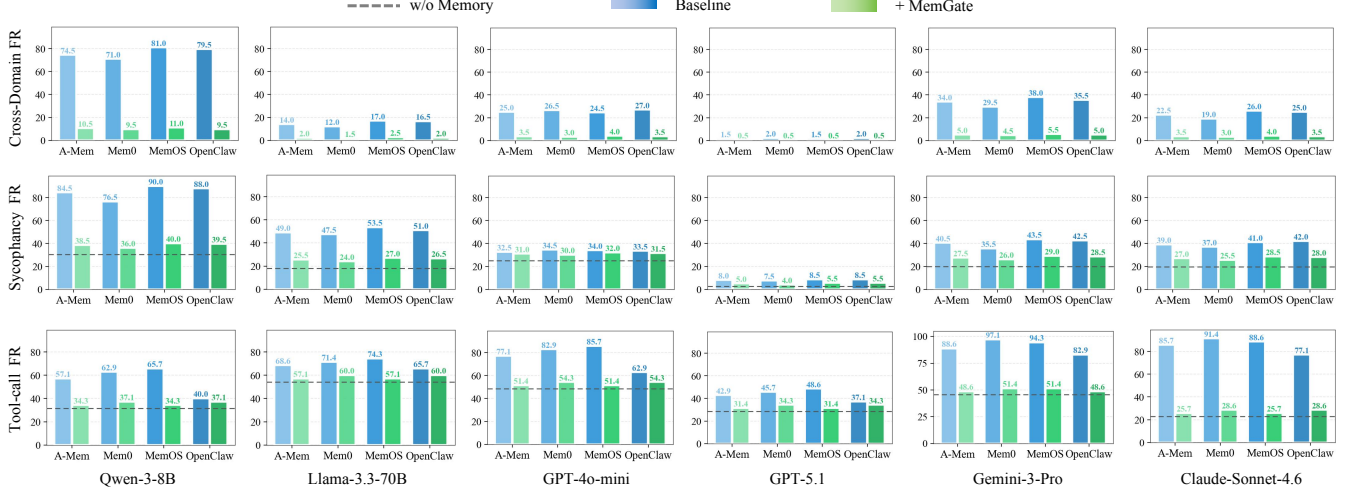


Figure 3: Performance of MemGate on over-personalization evaluations, including cross-domain leakage, sycophancy, and tool-call drift across agentic memory frameworks and the real-world agent.

Theorem 2 (Unsafe-Memory Suppression). Let $\ell_z(g^+, g^-)$ denote the DPO loss for a preference triple $z = (q, m^+, m^-)$ before applying the preservation penalty. Assume ℓ_z is L -smooth over a convex domain containing (g^+, g^-) and $(\mathbf{1}_d, \mathbf{1}_d)$. If optimization reaches a stationary point satisfying $0 \leq \ell_z(g^+, g^-) \leq \epsilon < \log 2$, the total gate perturbation is lower-bounded by:

$$\|\mathbf{1}_d - g^+\|_2^2 + \|\mathbf{1}_d - g^-\|_2^2 \geq \frac{2(\log 2 - \epsilon)}{L}.$$

In particular, if the positive memory is preserved ($g^+ = \mathbf{1}_d$), the unsafe-memory suppression gate g^- must satisfy:

$$\|\mathbf{1}_d - g^-\|_1 \geq \|\mathbf{1}_d - g^-\|_2 \geq \sqrt{\frac{2(\log 2 - \epsilon)}{L}}.$$

This theorem demonstrates that reducing the DPO loss below $\log 2$ forces the gates to deviate significantly from their reference identity states. Because the preservation penalty anchors g^+ near $\mathbf{1}_d$, this required perturbation is predominantly absorbed by g^- . This mathematically guarantees the suppression of unsafe memory dimensions. The complete proof is in Appendix B.

5. Experiments

In this section, we present a comprehensive empirical evaluation of MemGate. Specifically, our evaluation is designed to answer the following research questions:

- 1) Can MemGate improve the trustworthiness of agents?
- 2) Does MemGate preserve the utility of agents?
- 3) What is the runtime overhead of MemGate?
- 4) How do key hyperparameters affect MemGate?

5.1. Experimental Setup

MemGate training settings. We train MemGate as a lightweight gating module on top of frozen sentence embed-

dings. Queries and candidate memories are encoded with all-MiniLM-L6-v2 into 384-dimensional vectors. For each query-memory pair, the gate takes the concatenation of the query embedding, memory embedding, and their element-wise product as input, and outputs a sigmoid gate that masks only the memory embedding.

MemGate has about 9M parameters and is trained from scratch on 1,640 preference pairs. The preference pairs are synthetically constructed with GPT-4o-mini. To avoid test-set leakage, we construct these pairs from abstract memory-risk categories rather than from benchmark instances. They do not include any test queries, memory snippets, labels, or examples from the evaluation benchmarks. We optimize a DPO objective with the raw cosine retriever as the reference policy, together with a positive-memory mask preservation penalty weighted by $\lambda = 0.1$. The DPO temperature is 0.1 and $\beta = 0.3$. We use AdamW with learning rate 1.5×10^{-4} , batch size 256, and train for 20 epochs.

Benchmarks. For memory utility, we evaluate whether MemGate preserves the useful function of long-term memory. We use two general memory benchmarks, LoCoMo [33] and MemoryAgentBench [34], and three personalization-oriented benchmarks, PreFEval [35], PersonaMem [36], and the beneficial-memory subset of PersistBench [14]. LoCoMo and MemoryAgentBench measure long-term memory retrieval and reasoning, while PreFEval, PersonaMem, and PersistBench evaluate whether the agent can correctly use user preferences, profiles, and genuinely relevant memories. Detailed benchmark descriptions are provided in Appendix C.1.

5.2. Trustworthy Evaluation

We evaluate trustworthiness under two threat settings: benign over-personalization and malicious memory manipulation. For benign users, we measure cross-domain leakage,

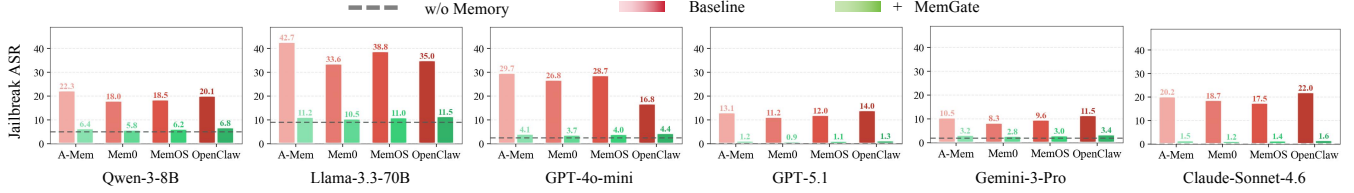


Figure 4: Performance of MemGate on memory-induced safety vulnerability evaluations across agentic memory frameworks and the real-world agent.

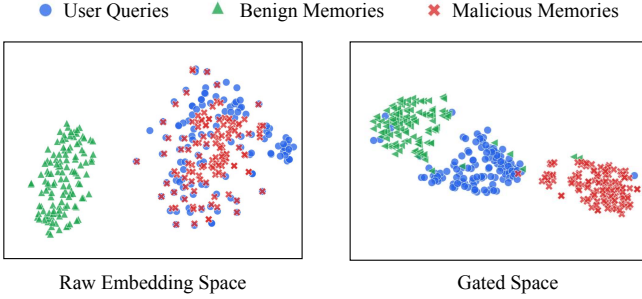


Figure 5: Representation shift induced by MemGate. In the raw embedding space, malicious memories overlap with user queries due to semantic similarity; after gating, malicious memories are separated in the gated subspace while benign memories remain close to the query cluster.

sycophancy, and tool-call drift. For malicious users, we measure memory-based jailbreak attack success.

As shown in Figure 3, MemGate consistently reduces over-personalization failures across agents and backbone models. In cross-domain leakage, memory-enabled agents may retrieve memories that are semantically related but contextually inappropriate, causing information from one domain to affect unrelated tasks. MemGate largely suppresses this misleading memory influence before generation. For example, on GPT-4o-mini, cross-domain FR drops from roughly 24.5%–27.0% to 3.0%–4.0% after applying MemGate.

Sycophancy is also mitigated, although not fully removed. MemGate reduces the memory-induced tendency to over-agree with stored user beliefs, while the remaining failures stay slightly above the memory-free baseline. This indicates that sycophancy is caused by both inappropriate memory activation and the base model’s intrinsic agreement bias. Tool-call drift also shows a clear improvement, but the memory-free baseline is higher than in the other risk categories because tool-parameter selection itself is challenging. Without MemGate, personal traits stored in memory can further bias safety-critical tool parameters such as validation mode, logging, rollback, approval gates, or execution policy. After applying MemGate, tool-call FR is reduced close to the memory-free level. On Claude-Sonnet-4.6, memory-enabled agents originally show high drift, ranging from 77.1% to 91.4%, while MemGate reduces it to 25.7%–28.6%, close to the w/o Mem result of 22.9%.

Figure 4 evaluates the malicious-user setting, where

planted memories are used to legitimize harmful requests. Without MemGate, persistent memory can become a durable attack surface: once a benign-looking context is stored, the agent may later use it to reinterpret a harmful query as legitimate. MemGate substantially reduces this vulnerability. On GPT-4o-mini, memory-enabled agents increase jailbreak ASR from the w/o Mem baseline of 2.5% to much higher levels, while MemGate brings it back to 3.7%–4.4%. Overall, MemGate provides the largest gains when failures are directly caused by inappropriate memory activation. Cross-domain leakage, tool-call drift, and memory-based jailbreaks are reduced substantially, while sycophancy remains more challenging due to the base model’s residual agreement bias.

To better understand how MemGate achieves this behavior, we visualize the embedding geometry before and after gating. We sample 120 user queries from PS-Bench, paired with 120 benign memories from LoCoMo dialogues and 120 malicious memories templated from the queries. All texts are encoded by all-MiniLM-L6-v2 into 384-dimensional vectors. We compare the raw L2-normalized embeddings with the gated embeddings obtained by element-wise multiplication with the gate vectors g_q and g_m , followed by L2 re-normalization. The vectors are projected to 2-D via t-SNE using perplexity 20, PCA initialization, and cosine distance. As shown in Figure 5, malicious memories strongly overlap with user queries in the raw embedding space because they are constructed to be semantically similar to the target request. After gating, the malicious memories are pushed away from the query cluster, while benign memories remain close to user queries. This representation-level separation shows that MemGate learns a task-aware subspace: it weakens adversarial or contextually misleading associations without collapsing the useful semantic neighborhood. This is consistent with the utility results, where MemGate improves memory reliability by reshaping retrieval toward safer and more relevant memory usage.

5.3. Utility Evaluation

Besides reducing memory-induced risks, MemGate should not sacrifice the core utility of memory. We therefore evaluate it on both general memory benchmarks and personalization-oriented benchmarks. Table 4 reports results on LoCoMo and MemoryAgentBench, while Table 6 reports results on PreFEval, PersonaMem, and the beneficial subset of PersistBench. MemGate preserves and slightly improves general memory utility across both Qwen-3-8B and GPT-

TABLE 4: Utility evaluation on general memory benchmarks. We report F1 score on LoCoMo and accuracy on MemoryAgentBench across different memory-enabled agents, with and without MemGate.

Agent	LoCoMo [33]					MemoryAgentBench [34]				
	Single-Hop	Multi-Hop	Open-Domain	Temporal	Overall	Single-Hop	Multi-Hop	Long-Context	Event	Overall
<i>Qwen-3-8B</i>										
A-Mem	43.7	37.4	18.6	15.9	37.3	18.8	24.3	29.1	30.4	25.7
+ MemGate	44.9	38.6	21.7	17.3	38.6	20.2	25.7	30.8	31.6	27.1
Mem0	48.5	44.5	16.7	12.5	40.0	22.4	27.6	32.3	33.1	28.8
+ MemGate	46.6	46.2	23.9	15.2	40.2	23.6	28.4	33.1	34.2	29.7
MemOS	50.7	46.1	18.4	14.6	41.7	24.1	30.3	34.8	35.6	31.2
+ MemGate	51.3	47.8	24.4	17.8	42.9	25.3	31.4	36.1	36.8	32.4
OpenClaw	44.6	38.1	21.9	17.5	38.5	20.7	26.1	31.2	31.8	27.5
+ MemGate	45.0	38.7	24.1	19.5	39.4	21.8	26.9	31.7	32.9	28.3
<i>GPT-4o-mini</i>										
A-Mem	41.3	35.6	20.1	33.4	37.8	22.7	28.2	33.1	35.4	29.9
+ MemGate	42.6	37.1	19.8	36.3	39.1	23.4	29.7	34.6	36.2	31.0
Mem0	48.6	42.1	15.1	35.3	42.9	25.0	32.0	36.0	37.5	32.6
+ MemGate	49.8	43.2	15.1	40.4	44.5	26.4	33.1	37.3	38.7	33.9
MemOS	50.4	44.2	17.3	38.9	45.4	27.8	35.3	39.2	40.7	35.8
+ MemGate	51.6	45.1	18.6	42.5	46.3	28.7	36.4	40.3	41.8	36.8
OpenClaw	42.5	36.9	22.8	35.1	38.9	24.2	30.7	35.4	36.1	31.6
+ MemGate	44.9	39.4	16.9	38.5	40.8	25.1	31.8	36.2	37.6	32.7

4o-mini. On LoCoMo, the overall scores remain stable after applying MemGate, with small gains in most agent settings. For example, on GPT-4o-mini, Mem0 improves from 42.9 to 44.5, while OpenClaw improves from 38.9 to 40.8. Similar trends appear on MemoryAgentBench, where MemGate consistently improves the overall score across all memory agents.

The utility improvement comes from MemGate’s selective reweighting rather than simple memory suppression. Conventional retrieval mainly relies on semantic similarity, which can retrieve memories that share surface-level keywords with the query but are not actually useful for the current task. Such noisy memories consume context budget and may distract the model from the truly relevant memory evidence. MemGate learns query- and memory-specific gates that down-weight misleading dimensions while preserving dimensions that are predictive of helpful memory use. As a result, the retrieved context becomes cleaner: irrelevant or adversarial memories are less likely to dominate the prompt, while task-relevant memories become easier for the model to use. This explains why MemGate can improve utility even though it is introduced primarily as a safety mechanism.

The personalization results in Table 6 further support this conclusion. MemGate improves PreFEval and PersonaMem accuracy in most settings, indicating that it does not weaken the agent’s ability to use genuinely helpful user preferences. More importantly, it substantially reduces Beneficial FR on PersistBench. For example, on GPT-4o-mini, OpenClaw reduces Beneficial FR from 14.3 to 6.7, while MemOS reduces it from 8.1 to 3.5. This suggests that MemGate does not block memory globally; instead, it improves the preci-

TABLE 5: Efficiency evaluation of MemGate. We report P50 (median latency) and P90 (90th-percentile latency) in seconds on GPT-4o-mini, together with LoCoMo F1 score.

Agent	P50 (s)	P90 (s)	LoCoMo F1
A-Mem	0.42	0.61	37.3
+ MemGate	0.44	0.68	38.6
Mem0	0.26	0.35	40.0
+ MemGate	0.29	0.41	40.2
MemOS	0.19	0.27	41.7
+ MemGate	0.23	0.30	42.9
OpenClaw	1.47	2.56	38.5
+ MemGate	1.59	2.75	39.4

sion of memory usage. By suppressing memories that are semantically tempting but contextually harmful, MemGate allows the model to rely more consistently on memories that are actually beneficial for the user query.

5.4. Efficiency

We further evaluate the runtime overhead introduced by MemGate during memory retrieval. Since MemGate is a lightweight gating module applied on top of frozen sentence embeddings, it does not require additional LLM calls or iterative reasoning. Its cost mainly comes from computing the query-memory gate and re-ranking memories in the gated embedding space.

Table 5 reports memory search latency on GPT-4o-mini. We report both P50 and P90 latency. P50 is the median

TABLE 6: Utility evaluation on personalization benchmarks. We report PreFEval accuracy, PersonaMem accuracy, and PersistBench beneficial failure rate across different memory-enabled agents, with and without MemGate.

Agent	PreFEval [35]	PersonaMem [36]	PersistBench [14]
	Accuracy↑	Accuracy↑	Beneficial FR↓
<i>Qwen-3-8B</i>			
A-Mem	72.9	44.2	26.1
+ MemGate	74.6	45.7	12.2
Mem0	77.2	50.9	19.6
+ MemGate	78.4	52.1	8.5
MemOS	79.5	54.6	16.2
+ MemGate	80.7	55.8	6.4
OpenClaw	75.0	47.6	23.3
+ MemGate	76.8	49.3	10.0
<i>GPT-4o-mini</i>			
A-Mem	84.3	52.1	16.4
+ MemGate	86.1	53.7	8.6
Mem0	88.0	56.8	11.4
+ MemGate	89.1	58.3	5.2
MemOS	89.6	61.2	8.1
+ MemGate	90.4	62.7	3.5
OpenClaw	86.4	54.6	14.3
+ MemGate	88.2	56.1	6.7

latency, meaning that half of the retrieval requests finish within this time. P90 is the 90th-percentile latency, meaning that 90% of the retrieval requests finish within this time. Compared with the average latency, P90 better reflects tail latency and captures slower retrieval cases that may affect user-perceived responsiveness in deployed agents.

Across all memory agents, MemGate introduces only a small latency increase. For example, Mem0 increases from 0.26s to 0.29s at P50. Even for OpenClaw, where the base memory search is already slower due to its system-level memory and tool integration, the additional overhead remains modest: P50 latency increases from 1.47s to 1.59s, and P90 latency increases from 2.56s to 2.75s.

Importantly, this small overhead does not come at the cost of utility. The overall LoCoMo score is preserved or slightly improved after applying MemGate across all evaluated agents. For instance, A-Mem improves from 37.3 to 38.6, MemOS improves from 41.7 to 42.9, and OpenClaw improves from 38.5 to 39.4. These results indicate that MemGate adds only lightweight computation while improving the quality of retrieved memories. Overall, MemGate provides a favorable efficiency–utility trade-off: it significantly improves trustworthiness with negligible additional retrieval latency.

5.5. Ablation Studies

We conduct ablation studies on GPT-4o-mini with OpenClaw to understand how MemGate’s key hyperparameters

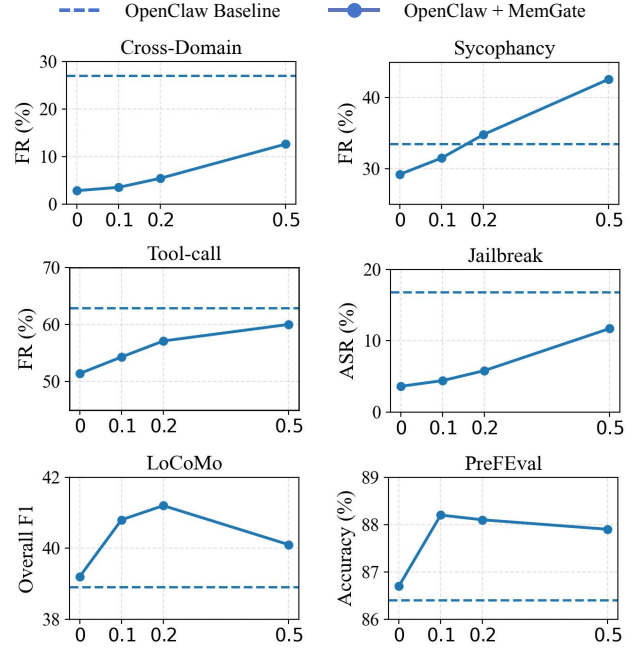


Figure 6: Ablation study of the regularization weight λ using GPT-4o-mini with OpenClaw. Lower values are better for failure rate (FR) and attack success rate (ASR), while higher values are better for Overall F1 and Accuracy.

affect the trade-off between trustworthiness and memory utility. We focus on two factors: the regularization weight λ used in MemGate training, and the number of retrieved candidate memories top- k .

Effect of regularization weight λ . Figure 6 shows the effect of varying λ from 0 to 0.5. A smaller λ makes the gate more conservative, suppressing irrelevant or risky memories more aggressively. As a result, cross-domain leakage, tool-call drift, and jailbreak ASR remain low when λ is small. However, overly conservative gating may also remove useful memory evidence, limiting the improvement on utility-oriented tasks.

A moderate value provides a better balance. In our main setting, $\lambda = 0.1$ keeps the trustworthy metrics low while improving LoCoMo and PreFEval over the OpenClaw baseline. When λ further increases, the gate becomes closer to an identity mapping and retains more memories, including noisy or harmful ones. This leads to higher cross-domain FR, sycophancy FR, tool-call FR, and jailbreak ASR. Meanwhile, utility first improves and then slightly drops, suggesting that excessive memory retention can also dilute useful evidence with irrelevant context.

Effect of retrieved memory size top- k . Figure 7 studies the impact of the number of retrieved memories. Increasing top- k can improve utility because more candidate memories enter the context, making it more likely that the model receives the evidence needed to answer memory-dependent queries. This explains why LoCoMo and PreFEval improve when top- k increases within a moderate range.

However, larger top- k also introduces more irrelevant

TABLE 7: Performance of MemGate with different embedding models on GPT-4o-mini with OpenClaw. The table compares trustworthiness, utility, model size, and retrieval latency under different MemGate encoders. Lower is better for FR, ASR, and latency, while higher is better for LoCoMo and PreFEval.

MemGate Setting			Trustworthy Evaluation				Utility Evaluation		Efficiency
MemGate Encoder	Dim.	Size	Cross-D. FR↓	Sycro FR↓	Tool-call FR↓	ASR↓	LoCoMo↑	PreFEval↑	Latency↓
<i>OpenClaw (baseline)</i>									
-	-	-	27.0	33.5	62.9	16.8	38.9	86.4	1.47s
<i>OpenClaw w. MemGate</i>									
all-MiniLM-L6-v2	384	35.1MB	3.5	31.5	54.3	4.4	40.8	88.2	1.59s
all-mpnet-base-v2	768	45.8MB	3.5	31.0	51.4	4.1	41.2	88.5	1.63s
bge-large-en-v1.5	1024	52.8MB	3.5	30.5	51.4	3.9	41.6	88.8	1.65s
text-embedding-3-small	1536	67.1MB	3.0	30.0	48.6	3.9	41.8	89.0	1.74s

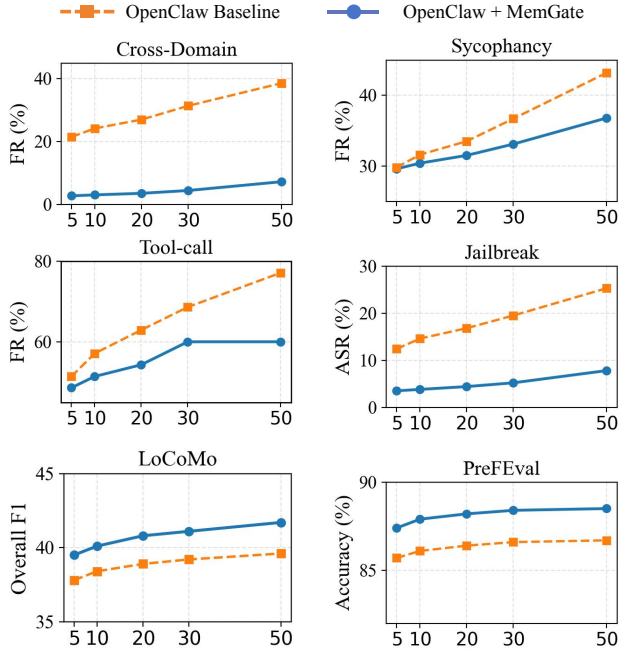


Figure 7: Top- k ablation study using GPT-4o-mini with OpenClaw. Lower values are better for failure rate (FR) and attack success rate (ASR), while higher values are better for Overall F1 and Accuracy.

or unsafe memories. For cross-domain leakage and sycophancy, this means more user profile facts or biased preferences may enter the prompt. For tool-call drift and jailbreak evaluation, larger candidate sets increase the chance that dangerous or misleading memories survive retrieval and influence the final response. Therefore, the trustworthy metrics generally worsen as top- k becomes too large. This result supports using different retrieval budgets for different evaluation goals: a slightly larger top- k is suitable for utility-oriented memory QA, while a smaller top- k is preferable for safety-sensitive settings.

Effect of MemGate encoder. We further study whether MemGate is tied to the default all-MiniLM-L6-v2 embedding space. Table 7 compares MemGate with four embedding encoders of different dimensionalities, ranging

from 384 to 1536. Larger embedding dimensions increase both the input size and the output gate size.

Overall, MemGate remains effective across all evaluated encoders. Compared with the OpenClaw baseline, every encoder substantially reduces cross-domain leakage, tool-call drift, and jailbreak ASR while improving LoCoMo and PreFEval. Stronger or higher-dimensional encoders bring small additional gains, but the improvement is incremental rather than structural. For example, replacing all-MiniLM-L6-v2 with text-embedding-3-small further reduces cross-domain FR from 3.5% to 3.0% and improves PreFEval from 88.2 to 89.0, but also increases the MemGate checkpoint size from 35.1MB to 67.1MB and fitted retrieval latency from 1.59s to 1.74s. This suggests that the main benefit comes from query-conditioned memory gating rather than a specific embedding model.

6. Conclusion

Long-term memory is becoming a core component of personal AI agents, our study shows that similarity-based memory retrieval can turn persistent personalization into a trustworthiness risk. Memories that are semantically close to a query may still be contextually inappropriate, causing cross-domain leakage, sycophancy, tool-call drift, and memory-induced jailbreaks. These failures arise because current agents often treat memory as a trusted user context without sufficiently deciding whether the memory should influence the current task.

We propose MemGate, a lightweight query-conditioned retrieval gate that re-ranks candidate memories before they enter the LLM context. By applying a task-conditioned mask to memory embeddings and training the gate with preference-aligned objectives, MemGate provides a modular way to improve contextual isolation without modifying the backbone model or invoking additional LLM calls. MemGate substantially reduces memory-induced failures while preserving, and in several cases improving, long-term memory utility. These results suggest that trustworthy personalization requires agents to not only remember user context, but also determine when memory is legitimate evidence for the current task.

References

- [1] Jiahong Liu, Zexuan Qiu, Zhongyang Li, Quanyu Dai, Wenhao Yu, Jieming Zhu, Minda Hu, Menglin Yang, Tat-Seng Chua, and Irwin King. A survey of personalized large language models: Progress and future directions. *arXiv preprint arXiv:2502.11528*, 2025.
- [2] Ranjan Sapkota, Konstantinos I Roumeliotis, and Manoj Karkee. Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. *Information Fusion*, page 103599, 2025.
- [3] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, et al. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*, 2025.
- [4] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [5] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604, 2026.
- [6] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, et al. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724*, 2025.
- [7] Yu Wang and Xi Chen. Mirix: Multi-agent memory system for llm-based agents. *arXiv preprint arXiv:2507.07957*, 2025.
- [8] Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. Search-ol: Agentic search-enhanced large reasoning models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 5420–5438, 2025.
- [9] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [10] Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. Seven failure points when engineering a retrieval augmented generation system. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*, pages 194–199, 2024.
- [11] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.
- [12] Edoardo DeBenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems*, 37:82895–82920, 2024.
- [13] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, 2024.
- [14] Sidharth Pulipaka, Oliver Chen, Manas Sharma, Taaha S Bajwa, Vyas Raina, and Ivaxi Sheth. Persistbench: When should long-term memories be forgotten by llms? *ICML*, 2026.
- [15] Jiahe Guo, Xiangran Guo, Yulin Hu, Zimo Long, Xingyu Sui, Xuda Zhi, Yongbo Huang, Hao He, Weixiang Zhao, Yanyan Zhao, et al. When personalization legitimizes risks: Uncovering safety vulnerabilities in personalized dialogue agents. *ACL*, 2026.
- [16] Sangyeon Yoon, Sunkyoung Kim, Hyesoo Hong, Wonje Jeung, Yongil Kim, Woosok Seo, Heuiyeon Yeen, and Albert No. Benchpres: A benchmark for context-aware personalized preference selectivity of persistent-memory llms. *arXiv preprint arXiv:2603.16557*, 2026.
- [17] Yulin Hu, Zimo Long, Jiahe Guo, Xingyu Sui, Xing Fu, Weixiang Zhao, Yanyan Zhao, and Bing Qin. Op-bench: Benchmarking over-personalization for memory-augmented personalized conversational agents. *arXiv preprint arXiv:2601.13722*, 2026.
- [18] Shuyue Stella Li, Bhargavi Paranjape, Kerem Oktar, Zhongyao Ma, Gelin Zhou, Lin Guan, Na Zhang, Sem Park, Lin Chen, Diyi Yang, et al. Horizonbench: Long-horizon personalization with evolving preferences. *arXiv preprint arXiv:2604.17283*, 2026.
- [19] Mahavir Dabas, Jihyun Jeong, Ming Jin, and Ruoxi Jia. Memory-induced tool-drift in llm agents. *arXiv preprint arXiv:2605.24941*, 2026.
- [20] Miao Yu, Fanci Meng, Xinyun Zhou, Shilong Wang, Junyuan Mao, Linsey Pan, Tianlong Chen, Kun Wang, Xinfeng Li, Yongfeng Zhang, et al. A survey on trustworthy llm agents: Threats and counter-measures. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 6216–6226, 2025.
- [21] OpenClaw Project. OpenClaw: Open-source personal AI agent framework. <https://openclaw.ai>, 2026.
- [22] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- [23] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher Manning. Raptor: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations*, volume 2024, pages 32628–32649, 2024.
- [24] Ahmad Al-Tawaha, Shangding Gu, Peizhi Niu, Ruoxi Jia, and Ming Jin. Remembering more, risking more: Longitudinal safety risks in memory-equipped llm agents. *arXiv preprint arXiv:2605.17830*, 2026.
- [25] Haoming Xu, Weihong Xu, Zongrui Li, Mengru Wang, Yunzhi Yao, Chiyu Wu, Jin Shang, Yu Gong, and Shumin Deng. When should models change their minds? contextual belief management in large language models. *arXiv preprint arXiv:2605.30219*, 2026.
- [26] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [27] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [28] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [29] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- [30] Google. Gemini documentation. <https://ai.google.dev/gemini-api/docs/gemini-3>, 2026.
- [31] Anthropic. Claude documentation. <https://docs.anthropic.com/>, 2026.
- [32] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- [33] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024.
- [34] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. *ICLR*, 2026.

- [35] Siyan Zhao, Mingyi Hong, Yang Liu, Devamanyu Hazarika, and Kaixiang Lin. Do llms recognize your preferences? evaluating personalized preference following in llms. *ICLR*, 2025.
- [36] Bowen Jiang, Zhuoqun Hao, Young-Min Cho, Bryan Li, Yuan Yuan, Sihao Chen, Lyle Ungar, Camillo J Taylor, and Dan Roth. Know me, respond to me: Benchmarking llms for dynamic user profiling and personalized responses at scale. *COLM*, 2025.

Appendix A.

Proof of Theorem 1

We provide the proof of Theorem 1 to formalize why the positive-mask preservation regularizer stabilizes useful memory representations. Since MemGate applies an element-wise mask to the memory embedding, the resulting score change can be bounded by first controlling the embedding perturbation and then relating it to the change in cosine similarity. This establishes that a smaller positive mask deficit leads to a smaller similarity drift for aligned memories.

Proof. Let $v'_{m+} = v_{m+} \odot g^+$. The representation change caused by the gate can be written as

$$v_{m+} - v'_{m+} = v_{m+} \odot (\mathbf{1}_d - g^+).$$

Thus, the magnitude of the perturbation is bounded by:

$$\begin{aligned} \|v_{m+} - v'_{m+}\|_2 &= \|v_{m+} \odot (\mathbf{1}_d - g^+)\|_2 \\ &\leq \|v_{m+}\|_\infty \|\mathbf{1}_d - g^+\|_1. \end{aligned}$$

We next relate this representation change to the change in cosine similarity. For a fixed $q \neq 0$, define the cosine function $f(v) = \frac{q \cdot v}{\|q\|_2 \|v\|_2}$. For any $v \neq 0$, its gradient with respect to v satisfies $\|\nabla f(v)\|_2 \leq \frac{1}{\|v\|_2}$. Consider the line segment connecting v'_{m+} and v_{m+} :

$$v_t = v'_{m+} + t(v_{m+} - v'_{m+}), \quad t \in [0, 1].$$

Since $g^+ \in [0, 1]^d$, each coordinate of v_t can be expressed as:

$$v_t = v_{m+} \odot (g^+ + t(\mathbf{1}_d - g^+)),$$

where the scaling coefficient is no smaller than g^+ coordinate-wise. Therefore, $\|v_t\|_2 \geq \|v'_{m+}\|_2 \geq \gamma$. Hence, f is $\frac{1}{\gamma}$ -Lipschitz continuous along this segment. By applying the mean value theorem, we obtain:

$$\begin{aligned} |S_{\text{MemGate}}(q, v_{m+}) - S(q, v_{m+})| &= |f(v'_{m+}) - f(v_{m+})| \\ &\leq \frac{1}{\gamma} \|v_{m+} - v'_{m+}\|_2 \leq \frac{\|v_{m+}\|_\infty}{\gamma} \|\mathbf{1}_d - g^+\|_1. \end{aligned}$$

This concludes the proof. Since our training objective explicitly penalizes the positive mask deficit, the regularizer mathematically limits the similarity drift of aligned memories, successfully discouraging MemGate from inadvertently suppressing useful memory evidence.

Appendix B.

Proof of Theorem 2

Proof. At the ungated reference initialization where $g^+ = g^- = \mathbf{1}_d$, the gated cosine similarities identically match the

ungated reference scores. Consequently, the relative margin inside the DPO objective evaluates to zero, yielding an initial loss of:

$$\ell_z(\mathbf{1}_d, \mathbf{1}_d) = -\log(\sigma(0)) = \log 2.$$

By the definition of L -smoothness, for any two vectors u and v within the convex domain, the function ℓ_z is upper-bounded by its quadratic expansion:

$$\ell_z(v) \leq \ell_z(u) + \langle \nabla \ell_z(u), v - u \rangle + \frac{L}{2} \|v - u\|_2^2.$$

Let $u = (g^+, g^-)$ and $v = (\mathbf{1}_d, \mathbf{1}_d)$. By the theorem's stationarity hypothesis, the gradient at the converged point is zero: $\nabla \ell_z(g^+, g^-) = \mathbf{0}$. Substituting these points into the smoothness condition yields:

$$\ell_z(\mathbf{1}_d, \mathbf{1}_d) \leq \ell_z(g^+, g^-) + \frac{L}{2} (\|\mathbf{1}_d - g^+\|_2^2 + \|\mathbf{1}_d - g^-\|_2^2).$$

Applying the established upper bounds $\ell_z(\mathbf{1}_d, \mathbf{1}_d) = \log 2$ and $\ell_z(g^+, g^-) \leq \epsilon$, we obtain the following inequality:

$$\log 2 \leq \epsilon + \frac{L}{2} (\|\mathbf{1}_d - g^+\|_2^2 + \|\mathbf{1}_d - g^-\|_2^2).$$

Rearranging this inequality provides the analytical lower bound on the total representation perturbation caused by the gating mechanism:

$$\|\mathbf{1}_d - g^+\|_2^2 + \|\mathbf{1}_d - g^-\|_2^2 \geq \frac{2(\log 2 - \epsilon)}{L}.$$

In the specialized scenario where the positive-memory preservation penalty perfectly anchors the aligned memory ($g^+ = \mathbf{1}_d$), the subtractive term vanishes. This immediately yields the final corollary bound for unsafe-memory suppression:

$$\|\mathbf{1}_d - g^-\|_1 \geq \|\mathbf{1}_d - g^-\|_2 \geq \sqrt{\frac{2(\log 2 - \epsilon)}{L}}.$$

This concludes the proof.

Appendix C.

Detailed Experiments Setups

C.1. Utility Benchmark Details

We evaluate utility from two complementary perspectives. The first group focuses on general long-term memory capability, measuring whether an agent can retrieve, maintain, and reason over information across long interaction histories. The second group focuses more specifically on personalization utility, measuring whether the system can correctly use beneficial user memories, preferences, or persona information when such memory is relevant to the current query.

General long-term memory benchmarks. These benchmarks evaluate whether a memory-enabled agent can preserve and use information over extended or incremental interactions.

- **LoCoMo.** LoCoMo [33] evaluates very long-term conversational memory over multi-session dialogues. It tests whether a model can recover and use information from extended interaction histories through tasks such as long-term question answering, event summarization, and dialogue generation. We report the overall F1 score.
- **MemoryAgentBench.** MemoryAgentBench [34] evaluates memory agents under incremental multi-turn interactions. It covers abilities such as accurate retrieval, long-range understanding, and selective forgetting. We report the overall accuracy.

Personalization benchmarks. In contrast to the above benchmarks, the following datasets emphasize whether the model can use user-specific memories when they are genuinely helpful. These benchmarks are therefore closer to the intended utility side of personalized memory systems: relevant memories should improve the response, while failure to retrieve or apply them reduces personalization quality.

- **PreFEval.** PreFEval [35] measures whether LLMs can infer, memorize, and follow user preferences in long-context conversational settings. It contains manually curated preference-query pairs across diverse topics and evaluates whether the model adapts responses according to previously expressed user preferences. We report accuracy.
- **PersonaMem.** PersonaMem [36] evaluates dynamic user profiling and personalized response selection across multi-session user–assistant interactions. Each instance contains evolving user profiles and interaction histories, requiring the model to select the response that best matches the user’s current persona and preferences. We report accuracy.
- **PersistBench beneficial subset.** PersistBench [14] contains both trustworthiness-oriented and utility-oriented memory tasks. For utility evaluation, we use its beneficial-memory subset, where long-term memory is genuinely relevant and should be used. We report beneficial failure rate (Beneficial FR), where a failure indicates that the system fails to retrieve or apply the necessary memory.